Script 13 — Execution Logger: User Guide

File: `scripts/13_log_execution.py`
Architecture: v3.2 (Feb 2026)
Role in pipeline: Post-execution bookkeeping — run *after* broker fills are confirmed

---

Table of Contents

1. What This Script Does
2. When to Run It
3. Prerequisites
4. The Three Operating Modes
   - Mode A — Single Trade
   - Mode B — Batch CSV
   - Mode C — Generate Template
5. The Recommended Monthly Workflow
6. Batch CSV Format
7. Validation Rules
8. Slippage Analysis
9. Output Files Reference
10. How Portfolio State is Updated
11. The Trade Ledger
12. Full CLI Reference
13. Common Errors and Fixes
14. Integration with Other Scripts
15. Audit Trail and Data Recovery

---

1. What This Script Does

Script 13 is the bridge between the *strategy* and the *broker*. After you execute the recommendations produced by Script 11, you use this script to record what actually happened — the real fill prices, the real share counts, the real commissions.

It does exactly four things, in order:

1. **Validates** every trade against ten business rules before touching any file.
2. **Updates** `data/portfolio_state.json` — the live portfolio record read by Scripts 09, 10, and 11.
3. **Appends** each trade as an immutable line to `data/trade_ledger.jsonl` — the permanent audit trail.
4. **Reports** per-trade slippage against the limit prices Script 11 recommended, and saves a session execution log.

It contains no strategy logic. It does not recalculate stops, re-rank momentum, or make decisions. It is purely a bookkeeping tool.

---

2. When to Run It

Trigger	Timing	Notes
Monthly rebalancing	After all fills on execution day (first trading day of month)	Use batch CSV mode
Intraday stop-loss hit	Same day as the stop fires and your broker executes	Use single-trade mode with `--action SELL`
Partial limit order fill	After session close if only part of a BUY filled	Log the actual filled shares and price
Unfilled limit order	No action needed	Rows with `fill_price = 0` are automatically skipped

Critical rule: always run Script 13 with all fills in hand. Do not run it speculatively or before your broker confirms execution. Partial session logging is supported — you can run the script twice in one day if fills arrive in batches.

---

3. Prerequisites

Before running Script 13, the following must be true:

- Script 11 has been run and produced `reports/rebalancing/{YYYY-MM}_recommendations.json`
- You have executed the recommended trades in your broker
- You have the broker confirmation showing actual fill prices, share counts, and commissions
- `data/portfolio_state.json` exists and reflects the portfolio state *before* today's trades (it was last updated by the prior Script 13 run)

No upstream scripts (01–11) need to be re-run before executing Script 13.

4. The Three Operating Modes

Mode A — Single Trade

Use this for one-off trades: an intraday stop-loss fill, a partial rebalancing fill that needs to be corrected, or any trade that falls outside the monthly batch.

Syntax:

```
python scripts/13_log_execution.py \
    --execution-date YYYY-MM-DD \
    --action         BUY|SELL \
    --symbol         TICKER.EXCHANGE \
    --shares         N \
    --fill-price     PRICE \
    --commission     AMOUNT \
    --broker-ref     "REF-STRING" \
    --notes          "Free text"
```

Example — recording a BUY fill:

```
python scripts/13_log_execution.py \
    --execution-date 2026-02-03 \
    --action         BUY \
    --symbol         AAPL.US \
    --shares         10 \
    --fill-price     152.45 \
    --commission     9.95 \
    --broker-ref     "ORD-20260203-001" \
    --notes          "Limit order filled at open +0.5%"
```

Example — recording a stop-loss SELL that fired intraday:

```
python scripts/13_log_execution.py \
    --execution-date 2026-01-15 \
    --action         SELL \
    --symbol         META.US \
    --shares         8 \
    --fill-price     398.20 \
    --commission     9.95 \
    --broker-ref     "SL-20260115-007" \
    --notes          "Stop-loss triggered, P1 exit"
```

Flags for single-trade mode:

Flag	Required	Type	Description
`--execution-date`	Yes	YYYY-MM-DD	Date the trade executed at the broker
`--action`	Yes	BUY or SELL	Trade direction
`--symbol`	Yes	string	Ticker with exchange suffix (e.g. `AAPL.US`)
`--shares`	Yes	integer	Number of whole shares
`--fill-price`	Yes	float	Actual fill price in EUR
`--commission`	No	float	Broker commission in EUR (default: 0.0)
`--broker-ref`	No	string	Broker order reference (for reconciliation)
`--notes`	No	string	Free-text annotation stored in ledger

Mode B — Batch CSV

Use this for the monthly rebalancing where you have multiple trades to record at once. This is the recommended workflow for all standard rebalancing sessions.

Syntax:

```
python scripts/13_log_execution.py \
    --execution-date 2026-02-03 \
    --batch-file     reports/executions/2026-02-03_fills.csv \
    --rebalancing-ref reports/rebalancing/2026-01_recommendations.json
```

The `--rebalancing-ref` flag is optional but strongly recommended — it enables per-trade slippage analysis against your Script 11 recommended prices.

What happens with unfilled rows:
If a limit order was not filled before end of day, leave `fill_price` as `0.0` in the CSV. The script will skip that row with a clear log message — it does not treat an unfilled order as an error. Re-evaluate that position in the next rebalancing cycle.

Mode C — Generate Template

This generates a pre-populated CSV from your Script 11 recommendations, so you have a ready-to-fill form on execution day.

Syntax:

```
python scripts/13_log_execution.py \
    --generate-template \
    --rebalancing-ref reports/rebalancing/2026-01_recommendations.json
```

What it produces:
A CSV at `reports/executions/{timestamp}_batch_template.csv` with one row per recommended action:

- All SELL rows (mandatory exits first, then rotation exits)
- All BUY rows (new entries, sorted by momentum rank)
- `fill_price` pre-set to `0.0` for you to fill in
- `notes` column pre-populated with the exit reason or entry rank

After execution day, fill in the actual prices and pass the file to Mode B.

---

5. The Recommended Monthly Workflow

This is the full step-by-step workflow for a standard end-of-month rebalancing session.

Step 1: Generate the template (day before execution)

```
python scripts/13_log_execution.py \
    --generate-template \
    --rebalancing-ref reports/rebalancing/2026-01_recommendations.json
```

This creates `reports/executions/{timestamp}_batch_template.csv`. Open it and review the pre-populated rows.

Step 2: Execute trades in your broker

Following the Script 11 execution checklist:

1. Execute all SELL orders first (market open)
2. Place BUY limit orders (close price + 0.5%)
3. Record each fill price, share count, and commission as fills arrive

Step 3: Fill in the template CSV

Open the template CSV and update:

- `fill_price` — the actual fill price from your broker confirmation
- `shares` — adjust if you received a partial fill
- `commission` — the actual commission charged
- `broker_ref` — the broker order reference (optional but useful for reconciliation)
- Leave `fill_price = 0.0` for any limit orders that were not filled

Step 4: Dry-run validation

Before writing any files, validate your CSV:

```
python scripts/13_log_execution.py \
    --execution-date 2026-02-03 \
    --batch-file     reports/executions/2026-02-03_fills.csv \
    --rebalancing-ref reports/rebalancing/2026-01_recommendations.json \
    --dry-run
```

Review the console output. Check for validation errors, duplicate warnings, and high-slippage alerts. Fix any issues in the CSV and re-run `--dry-run` until clean.

Step 5: Commit the execution

Remove `--dry-run` to write all outputs:

```
python scripts/13_log_execution.py \
    --execution-date 2026-02-03 \
    --batch-file     reports/executions/2026-02-03_fills.csv \
    --rebalancing-ref reports/rebalancing/2026-01_recommendations.json
```

Step 6: Verify outputs

Check that the following files were updated:

- `data/portfolio_state.json` — open it and confirm position count is correct
- `data/trade_ledger.jsonl` — check the last N lines match your trades
- `reports/executions/20260203_execution_log.csv` — review the slippage column

Step 7: Run Script 09 at end of week

New entries logged today will have `current_stop_price = null`. Script 09 must be run on the next Friday to calculate and set their initial stop-loss levels.

```
python scripts/09_calculate_stops.py \
    --as-of-date 2026-02-07 \
    --account-equity 52000
```

---

6. Batch CSV Format

Column definitions

Column	Required	Format	Description
`action`	Yes	`BUY` or `SELL`	Trade direction (case-insensitive)
`symbol`	Yes	`TICKER.EXCHANGE`	Must include exchange suffix
`shares`	Yes	integer	Whole shares only; decimals are truncated
`fill_price`	Yes	decimal	Actual broker fill price in EUR; use `0.0` for unfilled
`commission`	No	decimal	Broker commission in EUR; empty = `0.0`
`broker_ref`	No	string	Optional broker order reference
`notes`	No	string	Optional free-text; preserved in ledger

Example CSV

```
action,symbol,shares,fill_price,commission,broker_ref,notes
SELL,META.US,8,401.50,9.95,SL-001,EXIT-MANDATORY: stop_loss_hit
SELL,BABA.US,12,78.20,9.95,ROT-002,EXIT-ROTATION: rank=24
BUY,NVDA.US,5,875.30,9.95,ORD-003,ENTRY: rank=1 | limit≈873.85
BUY,MSFT.US,7,412.10,9.95,ORD-004,ENTRY: rank=3 | limit≈410.80
BUY,TSLA.US,0,0.0,0.0,,ENTRY: limit not filled - re-evaluate
```

Rules for the CSV

- The header row is required and must contain all column names (order does not matter)
- Column names are case-insensitive and leading/trailing whitespace is stripped
- Rows with `fill_price = 0` or blank `fill_price` are silently skipped (unfilled orders)
- At least one data row with a non-zero `fill_price` must be present, otherwise the script exits with a warning
- Commas inside a `notes` field must be quoted: `"EXIT: stop_loss, trend_reversal"`

---

7. Validation Rules

The script enforces ten validation rules before writing any file. All rules must pass — a single failure aborts the entire session with no changes written.

Rule	Code	What It Checks	Severity
Valid action	V1	`action` is `BUY` or `SELL`	Error
Symbol format	V2	Symbol is non-empty and contains a `.` (exchange suffix)	Error
Minimum shares	V3	`shares` ≥ 1	Error
Positive price	V4	`fill_price` > 0	Error
Non-negative commission	V5	`commission` ≥ 0	Error
Valid date	V6	`execution_date` parses as YYYY-MM-DD and is not in the future	Error
SELL has position	V7	For SELL: symbol exists in `portfolio_state.json`	Error
Share count	V8	For SELL: shares ≤ held shares	Error
Commission sanity	V9	Commission ≤ 5% of gross trade value	Error
Duplicate detection	V10	No matching trade exists in `trade_ledger.jsonl`	Error

Additionally, two warnings are raised but do not block execution:

- **Cost-averaging warning:** If you BUY a symbol that is already held in the portfolio, the script accepts the trade (computing a weighted average entry price) but logs a prominent WARNING. Dollar-cost averaging is non-standard for this strategy.

- **High commission warning:** If commission exceeds 1% of gross value, a non-blocking warning is displayed. This is separate from the V9 hard cap.

Understanding the duplicate check (V10)

Duplicates are identified by matching all five fields simultaneously: `execution_date`, `action`, `symbol`, `shares`, and `fill_price`. If `broker_ref` is provided, it is also matched. This means you can safely re-run the script on the same CSV (e.g., after a partial failure) — previously logged trades will be detected as duplicates and skipped. New trades in the same CSV will still be processed.

---

8. Slippage Analysis

When `--rebalancing-ref` is provided, the script cross-references each BUY trade against Script 11's recommended limit price and computes slippage.

How the recommended price is sourced

Script 11 recommends entries at: **close price + 0.5%** (the standard limit offset defined in `config/strategy_parameters.json`). The script looks for this value in the `entries.new[]` array of the recommendations JSON, under `limit_price` or `entry_limit_price`. If those fields are absent, it falls back to `close_price × 1.005`.

Slippage formulas

```
slippage_eur = (fill_price − recommended_price) × shares
slippage_pct = (fill_price − recommended_price) / recommended_price × 100
```

Interpretation

slippage_pct	Direction	Meaning
> 0%	**ADVERSE**	You paid more than the recommended limit price
= 0%	NEUTRAL	Exact fill at recommended price
< 0%	**FAVORABLE**	You paid less than the recommended limit price

A warning is printed in the console for any trade where `abs(slippage_pct) > 2.0%`. Persistent adverse slippage above 2% is a signal to review your order execution — either the limit offset needs adjusting in `strategy_parameters.json`, or orders are being placed at the wrong time.

Slippage is only computed for BUY trades. For SELLs, realized P&L is computed instead (see below).

Realized P&L on SELL trades

For every SELL, the script computes:

```
realized_pnl_eur = (fill_price − entry_price) × shares − commission
realized_pnl_pct = realized_pnl_eur / (entry_price × shares) × 100
```

`entry_price` is read from `portfolio_state.json`. If no entry price is recorded (e.g., manually entered position), P&L is shown as `n/a`.

---

9. Output Files Reference

Every Script 13 run produces or updates the following files.

Primary outputs (always written)

`data/portfolio_state.json`

The live portfolio record. Updated atomically after every successful run. Structure:

```
{
  "AAPL.US": {
    "entry_price":           152.45,
    "entry_date":            "2026-02-03",
    "shares":                10,
    "current_value":         1524.50,
    "unrealized_pnl":        0.0,
    "unrealized_pnl_pct":    0.0,
    "current_stop_price":    null,
    "initial_stop_price":    null,
    "trailing_stop_price":   null,
    "stop_type":             "initial",
    "stop_last_update_date": null,
    "data_last_update":      "2026-02-03T09:14:22.441",
    "is_new_entry":          true,
    "commission_entry":      9.95,
    "broker_ref_entry":      "ORD-20260203-001"
  }
}
```

Note that `current_stop_price` and `initial_stop_price` are `null` for new entries — these are set by Script 09 on the following Friday.

`data/trade_ledger.jsonl`

Append-only JSONL file. One JSON object per line. Never overwritten. Each line contains:

```
{
  "trade_id": "a3f9c12b4d6e",
  "logged_at": "2026-02-03T09:14:22.441Z",
  "action": "BUY",
  "symbol": "AAPL.US",
  "shares": 10,
  "fill_price": 152.45,
  "execution_date": "2026-02-03",
  "commission": 9.95,
  "broker_ref": "ORD-20260203-001",
  "notes": "Limit order filled at open +0.5%",
  "gross_value_eur": 1524.50,
  "net_value_eur": 1534.45,
  "recommended_price": 151.73,
  "slippage_eur": 7.20,
  "slippage_pct": 0.4738,
  "slippage_direction": "ADVERSE",
  "exit_reason": null,
  "realized_pnl_eur": null,
  "realized_pnl_pct": null
}
```

Session reports (written per execution date)

`reports/executions/{YYYYMMDD}_execution_log.json`

Full session report in JSON, including the post-execution portfolio snapshot:

```
{
  "metadata": {
    "execution_date": "2026-02-03",
    "total_trades": 6,
    "buy_count": 4,
    "sell_count": 2,
    "total_gross_eur": 12480.30,
    "total_commissions_eur": 59.70,
    "total_realized_pnl_eur": 842.15,
    "adverse_slippage_count": 2,
    "favorable_slippage_count": 1,
    "avg_slippage_pct": 0.2847,
    "warnings": [],
    "errors": []
  },
  "portfolio_after": { ... },
  "trades": [ ... ]
}
```

`reports/executions/{YYYYMMDD}_execution_log.csv`

Flat CSV with one row per trade for human review and spreadsheet analysis. Columns:

```
trade_id, execution_date, action, symbol, shares, fill_price,
gross_value_eur, commission, net_value_eur, recommended_price,
slippage_eur, slippage_pct, slippage_direction,
realized_pnl_eur, realized_pnl_pct, exit_reason, broker_ref, notes
```

Template file (only when `--generate-template` is used)

`reports/executions/{timestamp}_batch_template.csv`

Pre-populated from Script 11 recommendations. All `fill_price` values are `0.0` — ready for you to fill in after execution.

Backup files (automatically created)

`data/portfolio_state_backups/portfolio_state_{timestamp}.json`

A timestamped snapshot of `portfolio_state.json` taken immediately before every update. Created automatically — you do not need to manage these. They are your safety net if something goes wrong.

---

10. How Portfolio State is Updated

On a BUY

A new position entry is created in `portfolio_state.json` with:

- `entry_price` = actual fill price
- `entry_date` = execution date
- `shares` = filled share count
- `current_stop_price` = `null` (will be set by Script 09 on Friday)
- `is_new_entry` = `true`

If the symbol is **already held** (cost-averaging scenario), the shares are added to the existing position and `entry_price` is recalculated as the weighted average of old and new fills. A WARNING is logged because this is non-standard for this strategy.

On a SELL (full exit)

The position is **removed entirely** from `portfolio_state.json`. Scripts 09 and 10 will no longer process this symbol.

On a SELL (partial exit)

If `shares` < held shares, the position remains in `portfolio_state.json` with the remaining share count. The `entry_price` is preserved unchanged. A log line confirms how many shares remain.

---

11. The Trade Ledger

`data/trade_ledger.jsonl` is the permanent, immutable record of every trade executed in the strategy. It is used for:

- **Performance attribution** — reconstructing P&L per position over time
- **Slippage analysis over multiple months** — tracking execution quality trend
- **Compliance and audit** — a timestamped, UUID-keyed record of every action
- **Recovery** — if `portfolio_state.json` is ever corrupted, the ledger can be used to reconstruct it

Rules

- The ledger is **append-only**. The script never reads it to modify it — only to check for duplicates.
- Each line is a valid standalone JSON object.
- Each entry has a unique `trade_id` (12-character UUID fragment) and `logged_at` timestamp.
- Do not manually edit `trade_ledger.jsonl`. If you need to correct a mistake, add a reversing entry (a SELL at the same price as an erroneous BUY) and note it in the `notes` field.

Reading the ledger

To inspect or analyse the ledger in Python:

```
import json
from pathlib import Path

ledger = []
with open("data/trade_ledger.jsonl") as f:
    for line in f:
        if line.strip():
            ledger.append(json.loads(line))

# Example: all realized P&L from SELL trades
sells = [t for t in ledger if t["action"] == "SELL"]
total_pnl = sum(t["realized_pnl_eur"] for t in sells if t.get("realized_pnl_eur"))
print(f"Total realized P&L: €{total_pnl:,.2f}")

# Example: average slippage on BUY trades
buys_with_slip = [t for t in ledger if t["action"] == "BUY" and t.get("slippage_pct")]
avg_slip = sum(t["slippage_pct"] for t in buys_with_slip) / len(buys_with_slip)
print(f"Average BUY slippage: {avg_slip:+.4f}%")
```

---

12. Full CLI Reference

```
usage: 13_log_execution.py [-h]
    [--batch-file PATH | --generate-template]
    [--execution-date YYYY-MM-DD]
    [--action {BUY,SELL}]
    [--symbol TICKER.EXCHANGE]
    [--shares N]
    [--fill-price PRICE]
    [--commission EUR]
    [--broker-ref REF]
    [--notes TEXT]
    [--rebalancing-ref PATH]
    [--dry-run]
```

Flag	Mode	Default	Description
`--execution-date`	A, B	required	Trade execution date (YYYY-MM-DD). Applied to all trades in a batch.
`--action`	A	required	`BUY` or `SELL` (case-insensitive)
`--symbol`	A	required	Ticker with exchange suffix (e.g. `NVDA.US`, `VOW3.XETRA`)
`--shares`	A	required	Whole number of shares executed
`--fill-price`	A	required	Actual broker fill price in EUR
`--commission`	A	`0.0`	Broker commission in EUR
`--broker-ref`	A	`""`	Broker order reference string
`--notes`	A	`""`	Free-text note appended to ledger entry
`--batch-file`	B	—	Path to a CSV file with multiple fills
`--generate-template`	C	—	Generate blank CSV from Script 11 output
`--rebalancing-ref`	B, C	auto-detect	Path to Script 11 recommendations JSON
`--dry-run`	A, B	`false`	Validate and report without writing any files

Mode flags `--batch-file` and `--generate-template` are mutually exclusive — you cannot use both in the same invocation.

---

13. Common Errors and Fixes

`V7: SELL on 'AAPL.US' but no position found in portfolio_state.json`

Cause: You are trying to SELL a symbol that is not currently recorded as held.

Fix options:

- Check for a typo in the symbol (e.g., `AAPL.US` vs `AAPL.LSE`)
- Check that the prior BUY was correctly logged — if it was not, log the BUY first with the correct `--execution-date`
- If the position was acquired before the ledger was set up, add it manually to `portfolio_state.json` before running Script 13

`V8: SELL 15 shares of 'MSFT.US' but only 10 held`

Cause: The shares in your CSV exceed what `portfolio_state.json` records as held.

Fix: Correct the `shares` value in your CSV. If the true held count is higher than what the state file shows, your state file is out of sync — reconstruct it from `trade_ledger.jsonl` or fix it manually, then re-run.

`V9: Commission 1500.00 EUR is 12.4% of gross value — exceeds 5% sanity cap`

Cause: Almost always a data entry error — commission entered in the `fill_price` column, or fill price entered in the `commission` column.

Fix: Swap the values in your CSV and re-run.

`DUPLICATE: A matching trade for AAPL.US already exists in the ledger`

Cause: You are running the same batch CSV for a second time (perhaps after fixing a different error).

Behaviour: Duplicate rows are **skipped automatically** — the script continues processing other rows. This is intentional and safe. You do not need to remove the duplicate row from your CSV before re-running.

When to investigate: If you see this for a trade you did not intend to duplicate, check the ledger to confirm only one entry exists: `grep "AAPL.US" data/trade_ledger.jsonl`

`Batch CSV is missing required columns: ['fill_price']`

Cause: Your CSV has a different column name (e.g., `fill price` with a space, or `FillPrice`).

Fix: Column names are case-insensitive but must match exactly (no spaces). Required columns are: `action`, `symbol`, `shares`, `fill_price`, `commission`, `broker_ref`, `notes`.

`execution_date '2026-02-30' is not a valid YYYY-MM-DD date`

Cause: The date does not exist (February 30 is not a real date) or is formatted incorrectly.

Fix: Use a valid calendar date in `YYYY-MM-DD` format. Note: the script does not validate whether the date is a trading day — that is your responsibility.

Portfolio state shows wrong share count after a partial fill

Scenario: Script 11 recommended you buy 10 shares of `NVDA.US`, but your broker only filled 6 shares.

Correct approach: Log the 6 shares that were actually filled:

```
python scripts/13_log_execution.py \
    --execution-date 2026-02-03 \
    --action BUY \
    --symbol NVDA.US \
    --shares 6 \
    --fill-price 875.30 \
    --notes "Partial fill: 6 of 10 shares. Re-evaluate remaining 4 in next session."
```

The remaining 4 shares are not yet held and should not be logged. If your broker fills them in a subsequent session, log that as a separate BUY trade on the correct date. The script will detect the cost-averaging scenario and compute a weighted average entry price automatically (with a WARNING).

`No Script 11 recommendations file found — slippage analysis skipped`

Cause: `--rebalancing-ref` was not supplied and no file was found in `reports/rebalancing/`.

Effect: Non-critical. All other functionality works normally. Slippage fields in the ledger will be `null`.

Fix: Explicitly pass the path: `--rebalancing-ref reports/rebalancing/2026-01_recommendations.json`

---

14. Integration with Other Scripts

Script 13 sits at the end of the pipeline. It writes outputs consumed by several upstream scripts on the *next* cycle.

```
Script 11 (Monthly Rebalancer)
    → Human executes in broker
    → Script 13 (Execution Logger)       ← You are here
        → data/portfolio_state.json
            → Script 09 (next Friday)    — calculates initial stops
            → Script 10 (daily/monthly)  — checks exit conditions
            → Script 11 (next month)     — identifies holds and rotation
        → data/trade_ledger.jsonl
            → performance attribution    — external analysis
        → reports/executions/
            → Script 14 (daily monitor)  — consumes for position verification
```

Dependency summary

Writes to	Read by	When
`portfolio_state.json`	Script 09	Next Friday (stop calculation)
`portfolio_state.json`	Script 10	Daily / pre-rebalancing
`portfolio_state.json`	Script 11	Monthly rebalancing
`trade_ledger.jsonl`	External analytics	As needed
`execution_log.json`	Script 14	Next trading day

---

15. Audit Trail and Data Recovery

Backup strategy

Every time Script 13 writes `portfolio_state.json`, it first saves a timestamped backup to `data/portfolio_state_backups/portfolio_state_{YYYYMMDD_HHMMSS}.json`. These backups are never deleted by the system — manage disk space manually if needed (monthly backups are small; a year of monthly runs produces ~12 files of a few KB each).

Recovering from a corrupted portfolio state

If `portfolio_state.json` is corrupted or accidentally deleted:

1. Find the most recent clean backup: `ls -lt data/portfolio_state_backups/`
2. Copy it back: `cp data/portfolio_state_backups/portfolio_state_{timestamp}.json data/portfolio_state.json`
3. Identify which Script 13 runs occurred after that backup using `data/trade_ledger.jsonl`
4. Re-run those Script 13 invocations in chronological order using single-trade mode, sourcing values from the ledger

Since the ledger has duplicate detection, re-running will skip already-logged trades and only apply missing ones.

Reconstructing portfolio state from scratch

In an extreme case where both `portfolio_state.json` and all backups are lost:

```
import json
from pathlib import Path
from datetime import datetime

ledger_path = Path("data/trade_ledger.jsonl")
portfolio   = {}

with open(ledger_path) as f:
    for line in sorted(f, key=lambda l: json.loads(l)["execution_date"]):
        trade = json.loads(line.strip())
        sym   = trade["symbol"]

        if trade["action"] == "BUY":
            if sym in portfolio:
                held = portfolio[sym]["shares"]
                new  = trade["shares"]
                old_price = portfolio[sym]["entry_price"]
                avg_price = (old_price * held + trade["fill_price"] * new) / (held + new)
                portfolio[sym]["shares"]      = held + new
                portfolio[sym]["entry_price"] = round(avg_price, 4)
            else:
                portfolio[sym] = {
                    "entry_price": trade["fill_price"],
                    "entry_date":  trade["execution_date"],
                    "shares":      trade["shares"],
                }
        elif trade["action"] == "SELL":
            if sym in portfolio:
                remaining = portfolio[sym]["shares"] - trade["shares"]
                if remaining <= 0:
                    del portfolio[sym]
                else:
                    portfolio[sym]["shares"] = remaining

with open("data/portfolio_state.json", "w") as f:
    json.dump(portfolio, f, indent=2)

print(f"Reconstructed {len(portfolio)} positions from ledger.")
```

Note: this reconstruction will not contain stop-loss fields — re-run Script 09 after recovery to restore stop levels.

Architecture v3.2 — Last updated Feb 2026